

1 SaaS Platform Architecture

To transition the deep learning models into an industrial service, a Software-as-a-Service (SaaS) platform was developed. The design strictly prioritises decoupling, multi-user concurrency, and data security.

1.1 Architecture Overview and Data Isolation

The system adopts a decoupled four-tier architecture (Figure 1), isolating presentation, routing (Flask), execution (`ThreadPoolExecutor`), and relational persistence (SQLAlchemy).

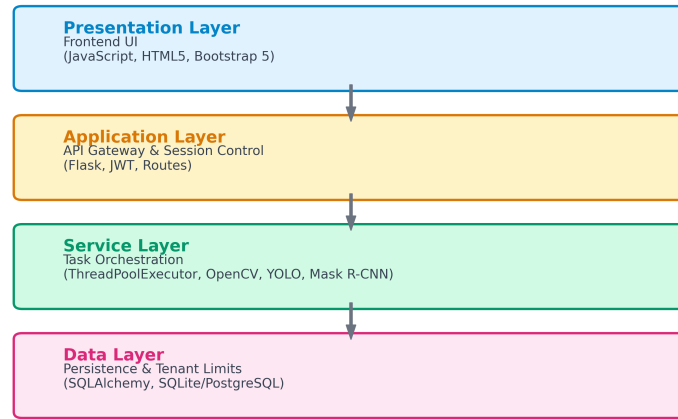


Figure 1: Four-tier SaaS architecture.

To accommodate multiple power grid subsidiaries, strict Entity-Relationship boundaries apply. Every `InferenceTask` inherits both a `user_id` and a `tenant_id`. Security is enforced at the routing layer via interceptors, appending `tenant_id` filters to all queries rather than relying on implicit ORM hooks.

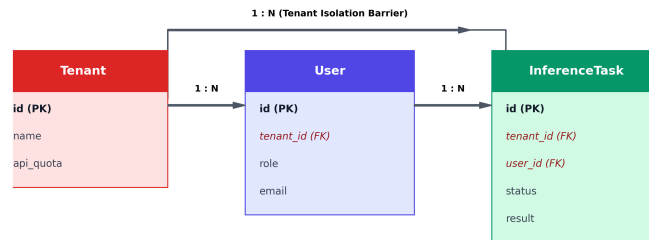


Figure 2: Data isolation constraints.

1.2 Asynchronous Execution and Model Registry

Synchronous inference mechanisms exhaust Web frameworks under concurrent load. Ergo, an event-driven task pipeline (`PENDING` $\rightarrow$ `RUNNING` $\rightarrow$ `SUCCESS`) was designed. Client

submissions immediately return HTTP 202, while memory queues instantaneously awaken background worker threads, eliminating database polling bottlenecks.

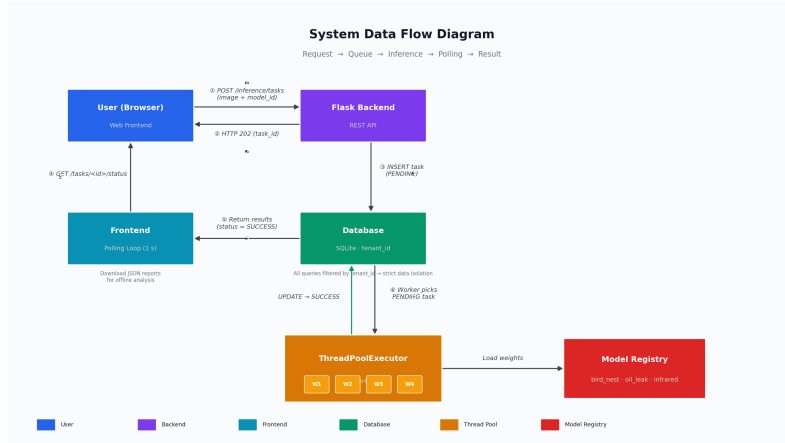


Figure 3: Event-driven asynchronous lifecycle.

Additionally, to prevent 12 GB hardware resource exhaustion from simultaneously loading YOLOv8s and Mask R-CNN, an Adapter Pattern Model Registry defers memory allocation. Weights are exclusively *lazy-loaded* upon their first explicit invocation and cached thereafter.